

Performance Testing

Date	29 September 2026
Team ID	SWTID-2026-7019
Project Name	EduGenie – AI Learning Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python-based load testing tool) – script: locustfile.py
Type of Testing	Load Testing (baseline and expected load), Stress Testing, and Spike Testing
Target Module / API	EduGenie FastAPI backend (main.py): GET /qa (Q&A), POST /explain/ (topic explanation), POST /summarize/ (text summarization), POST /quiz (quiz generation), GET /learn/recommendations (learning path). GET / (frontend page) is used as a baseline.
Test Environment	Local – FastAPI app served by Uvicorn on http://127.0.0.1:8000 (Windows machine, Python 3.13); AI responses served by the Google Gemini API
Test Date	29 September 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline load – light mixed traffic across all endpoints	10	60	Avg response < 2 s, max < 5 s; error rate < 1%
2	Expected load – normal classroom usage, mixed requests with 1–3 s think time	25	120	System stays stable; error rate < 1%; CPU and memory < 80%
3	Stress test – users ramped up gradually (5 users/sec) to find the breaking point	50	180	Find the load where errors begin; server does not crash

 4	SMARTBRIDGE Let's Bridge the Gap Spike test – sudden jump from 5 to 100 users on /qa and /explain/	100	60	Server recovers after the spike without restart
--	--	-----	----	---

Step 3: Performance Test Results

					
S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	[Fill after test]	[Fill after test]	
2	Response Time (Max)	< 5 seconds	[Fill after test]	[Fill after test]	
3	Throughput (Req/sec)	> 5 req/sec	[Fill after test]	[Fill after test]	
4	Error Rate	< 1%	[Fill after test]	[Fill after test]	
5	CPU Utilization	< 80%	[Fill after test]	[Fill after test]	
6	Memory Utilization	< 80%	[Fill after test]	[Fill after test]	

Step 4: Observations & Analysis

Key Findings:

[Describe what the results revealed once the Locust run is complete – e.g. the user count at which average response time crossed 2 s, the point where errors started, and how the server behaved during and after the spike.]

Bottlenecks Identified:

Identified from a review of the code; confirm each one against your test data.

- Blocking calls inside async routes: every endpoint is declared `async def` but calls the synchronous Gemini SDK (`client.models.generate_content()`), which blocks the event loop and makes concurrent users wait in a queue.
- A new `genai.Client` is created on every request, adding avoidable overhead.
- External API dependency: response time is dominated by the Gemini API, and free-tier rate limits (HTTP 429) are likely to cause failures under stress and spike loads.
- Local explanation model: when the `LaMini-Flan-T5-783M` model loads successfully, `/explain/` runs CPU inference in-process, which is heavy on CPU and memory.
- Errors are returned as HTTP 200 with an "[ERROR] ..." string, so failures can be hidden from the tool unless the response body is checked (the Locust script does this).
- No caching of repeated questions/topics and no timeout or rate limiting on requests.

Optimization Steps Taken:

Recommended changes – tick off or edit these to reflect what you actually applied.

- Switch to the async Gemini client (`client.aio`) or run SDK calls with `run_in_threadpool` so the event loop is not blocked.
- Create one shared `genai.Client` at startup and reuse it.
- Add an in-memory cache for repeated `/qa/`, `/explain/` and `/learn/recommendations` requests.
- Set request timeouts and retry with backoff on 429 errors; run Uvicorn with multiple workers.

Step 5: Screenshots / Evidence



Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here – Locust statistics table]

[Insert Screenshot 2 here – Locust charts (users, response times, RPS)]